CHAPITRE 2

Programmation orientée objet en Java

- Classes, objets, attributs et méthodes
- Héritage et redéfinition
- Packages et organisation du code
- Constructeurs, encapsulation, this / static / final
- Polymorphisme, classes abstraites, interfaces

Définir une classe et créer des objets

```java
public class Etudiant {

    private String matricule;
    private String nom;
    private double moyenne;

    public Etudiant(String matricule, String nom) {
        this.matricule = matricule;
        this.nom = nom;
    }

    public String getNom() {
        return nom;
    }
}

// Utilisation
Etudiant e1 = new Etudiant("E001", "Amine");
System.out.println(e1.getNom());
```

Concepts de la POO

- Définition : Paradigme de programmation basé sur les objets qui encapsulent des données et des comportements.
- Principes fondamentaux :
  - Encapsulation : Regrouper les données et les méthodes dans une classe.
  - Héritage : Réutiliser et étendre les fonctionnalités existantes.
  - Polymorphisme : Adapter le comportement en fonction du contexte.
  - Abstraction : Cacher les détails complexes pour ne montrer que l'essentiel.

Vocabulaire objet

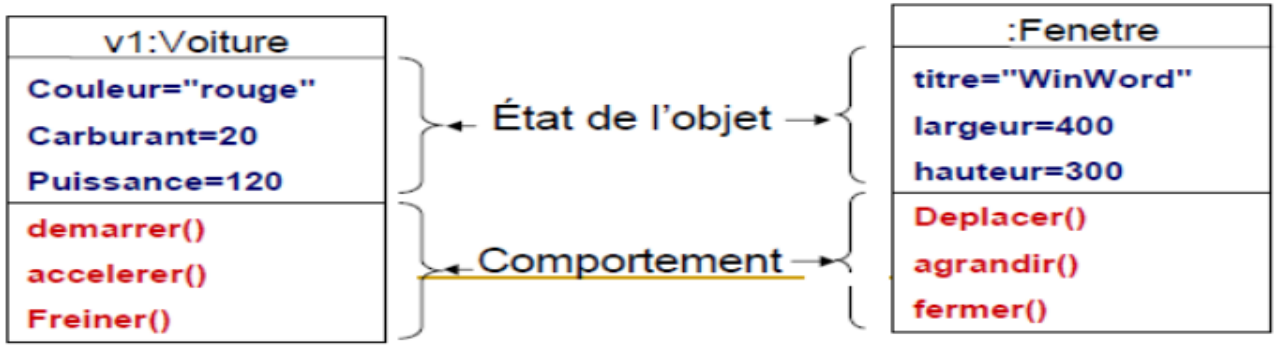
- Classe : modèle/plan décrivant un ensemble d'objets (attributs + comportements)
- Objet (instance) : élément concret créé à partir d'une classe via new
- Attribut : donnée propre à chaque objet (état)
- Méthode : comportement/action que l'objet peut exécuter
- Chaque objet possède sa propre copie des attributs d'instance

Définir une classe et créer des objets

Objet

- **Un objet est une structure informatique définie par un état et un comportement.**
- **Objet = état + comportement.**
- **L'état regroupe les valeurs instantanées de tous les attributs de l'objet.**
- **Le comportement regroupe toutes les actions et les réactions de l'objet.**
- **L'état d'un objet peut changer dans le temps.**
- **Généralement , c'est le comportement qui modifie l'état de l'objet.**

Exemple:



Définir une classe et créer des objets

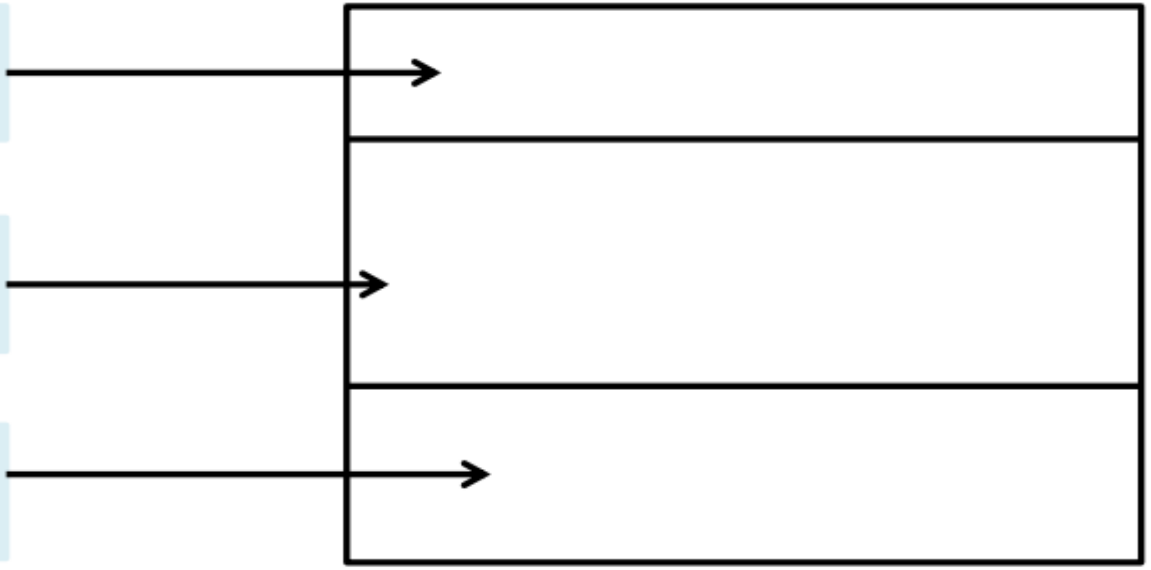
Classe

- **Une classe est généralement représenté par un rectangle qui comporte trois compartiments**

Compartiment 1 : Nom de la classe qui doit commencer par une majuscule.

Compartiment 2 : Les attributs.

Compartiment 3: les méthodes



Définir une classe et créer des objets

Classe

Attribut:

- **Les attributs et les opérations sont les propriétés d'une classe. Les attributs décrivent l'état de l'objet de la classe. Un attribut est déclaré selon le format suivant:**

Visibilté TypeAttribut nomAttribut [Multiplicité] = [ValeurInit]

- **Visibilité: + (Public), - (Privé), # (Protégé), par défaut la Visibilté est protected.**
- **nomAttribut: Le nom de l'attribut doit être unique et commencer par un minuscule.**
- **Multiplicité: nombre d'attributs de ce type (Exemple tableau []).**
- **TypeAttribut: le type de l'attribut (type primitif ou classe).**
- **ValeurInit: la valeur initiale de l'attribut.**

Définir une classe et créer des objets

Classe

Méthode:

- **Les méthodes décrivent le comportement des objets. Elles représentent les fonctions qu'une classe sait exécuter.**
- **Les méthodes appelées également méthodes dans les langages objets.**
- **Syntaxe UML pour la définition d'une opération:**

Visibilté TypeDeRetour nomOperation (Paramètres)

- ❑ **Visibilté: + (public), - (privée) ou # (protégée).**
- ❑ **TypeDeRetour: le type de la valeur de retour de l'operation (type primitif ou classe)**
- ❑ **nomOperation: nom de la méthode**
- ❑ **Paramètres: les paramètres de la méthode.**

Définir une classe et créer des objets

Encapsulation (Accessibilité):

Dans java, il existe 4 niveaux de la protection:

- **Private (-) : Un nombre privé d'une classe n'est accessible qu'a l'intérieur de cette classe.**
- **Protected (#) : un membre protégé d'une classe est accessible à :**
  - **L'intérieur de la classe.**
  - **Aux classes dérivées de cette classe.**
  - **Aux classes de même package.**
- **Public (+): accès à partir de toute entité interne et externe à la classe.**

RQ : Par défaut l'autorisation est de package. Cette autorisation indique que uniquement les classes du même package ont l'autorisation d'accès.

Constructeurs

1

Rôle du constructeur

Méthode spéciale appelée automatiquement lors de la création d'un objet (new), pour l'initialiser.

2

Même nom que la classe, sans type de retour

public Etudiant(String matricule, String nom) { ... }

3

Constructeur par défaut

Si aucun constructeur n'est écrit, Java en fournit un vide automatiquement (attributs à leur valeur par défaut).

4

Surcharge de constructeurs

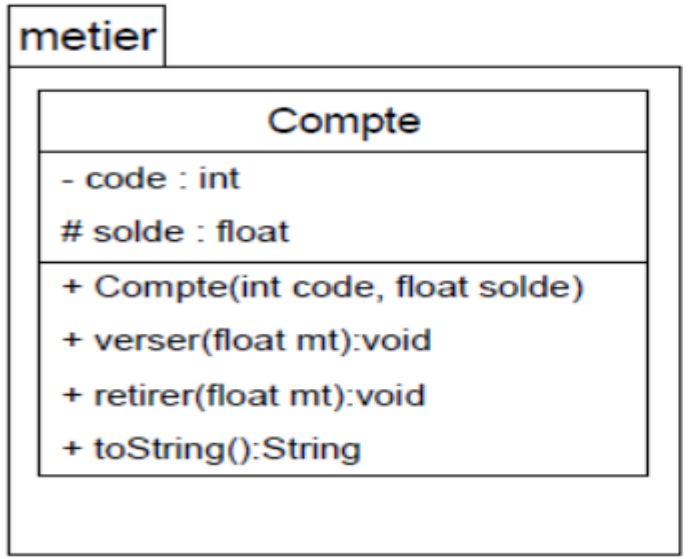
Plusieurs constructeurs possibles avec des paramètres différents, pour offrir plusieurs façons de créer un objet.

5

Appel entre constructeurs : this(...)

Un constructeur peut en appeler un autre de la même classe avec this(...), en première instruction.

Exemple d'implementation d'une classe avec java




```java
package metier;
public class Compte {
// Attributs
private int code;
protected float solde;
// Constructeur
public Compte(int c,float s){
    code=c;
    solde=s;
}
// Méthode pour verser un montant
public void verser(float mt){
    solde+=mt;
}
// Méthode pour retirer un montant
public void retirer(float mt){
    solde-=mt;
}
// Une méthode qui retourne l'état du compte
public String toString(){
    return(" Code="+code+" Solde="+solde);
}
}
```

Création des Objets dans java:

- **Dans java, pour créer un objet d'une classe, on utilise la commande new suivie par un constructeur de la classe.**
- **Les constructeurs de la classe sont des méthodes spéciales d'une Visibilté public, qui porte le même nom que la classe et ils ont pas un type de retour.**
- **Il existe plusieurs types des constructeurs:**
  - ❑ **Constructeur par défaut: toute classe à un constructeur par défaut qui ne reçoit pas des paramètres .**
  - ❑ **Constructeur avec des paramètres: définit par le programmeur s'il est nécessaire. Exemple**

```
public etudiantIcda(String nom, String prenom) {
    super();
    this.nom = nom;
    Prenom = prenom;
}
```

Création des Objets dans java:

- **Constructeur synthétisé: une fois le programmeur déclare un constructeur avec paramètres le constructeur par défaut n'est plus exploitable, alors le programme déclare un constructeur vide nommé synthétisé.**

- **Exemple de la syntaxe:**

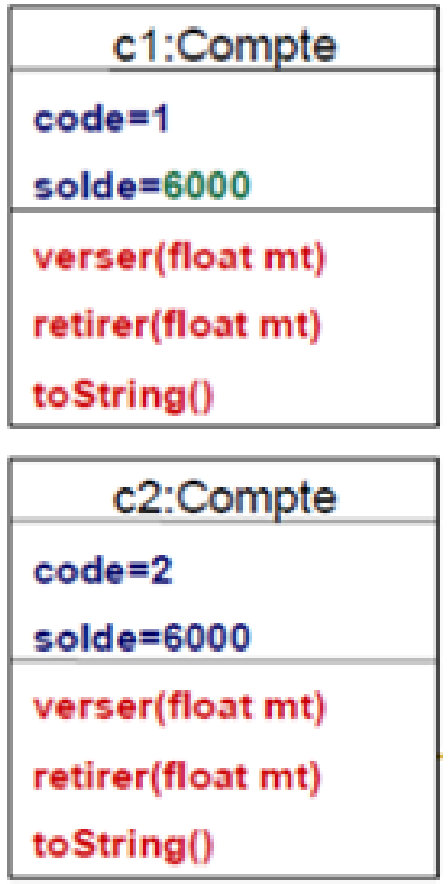
```
public static void main(String[] args) {
  Compte c1=new Compte(1,5000);
  Compte c2=new Compte(2,6000);
  c1.verser(3000);
  c1.retirer(2000);
  System.out.println(c1.toString());
}
```

C1 : code = 1, Solde =5000

C2 : code = 2, Solde =6000

C1 : code = 1, Solde = 8000 : 5000+3000

Code=1 Solde= 6000



Encapsulation et modificateurs d'accès

Dans java, il existe 4 niveaux de la protection:

- **Private (-) : Un nombre privé d'une classe n'est accessible qu'a l'intérieur de cette classe.**
- **Protected (#) : un membre protégé d'une classe est accessible à :**
  - **L'intérieur de la classe.**
  - **Aux classes dérivées de cette classe.**
  - **Aux classes de même package.**
- **Public (+): accès à partir de toute entité interne et externe à la classe.**

RQ : Par défaut l'autorisation est de package. Cette autorisation indique que uniquement les classes du même package ont l'autorisation d'accès.

Encapsulation et modificateurs d'accès

Encapsulation (Accessibilité): Getters et Setters

- **Les attributs privés d'une classe ne sont pas accessibles qu'a l'intérieur de la classe.**
- **Pour donner la possibilité à autres classes d'accéder à ces attributs il faut définir dans la classe des méthodes publiques qui permettant de:**
  - ❑ **Lire la variable: accesseurs ou Getters.**
  - ❑ **Modifier la variable: mutateurs ou Setters.**
- **Exemple de la déclaration des Getters:**

```
public String getNom() {
    return nom;
}
```

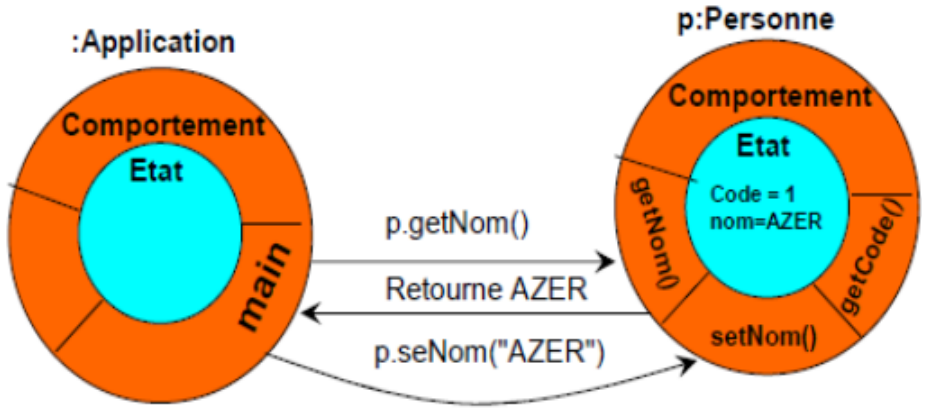
- **Exemple de la déclaration des Setters:**

```
public void setNom(String nom) {
    this.nom = nom;
}
```

Encapsulation et modificateurs d'accès

Encapsulation (Accessibilité):

```
public class Application {
public static void main(String[] args) {
  Personne p=new Personne();
  p.setNom("AZER");
  System.out.println(p.getNom());
}
}
```



- **Généralement l'état d'un objet est privé ou protégé et son comportement est publique.**
- **Quand l'état de l'objet est privé Seules ces méthodes qu'ont le droit d'y accéder.**
- **Quand l'état de l'objet est protégé, les méthodes des classes dérivées et les classes appartenant au même package peuvent également y accéder .**

Encapsulation et modificateurs d'accès

```
public class Etudiant {
    private String nom;           // accessible uniquement
dans la classe
    private double moyenne;

    public String getNom() {           // accesseur
(getter)
        return nom;
    }

    public void setMoyenne(double m) { // mutateur
(setter) avec contrôle
        if (m >= 0 && m <= 20) {
            this.moyenne = m;
        } else {
            throw new IllegalArgumentException("Moyenne
invalide");
        }
    }
}
```

Pourquoi encapsuler ?

- ✦ private protège les attributs : accès uniquement via getters/setters
- ✦ Permet de valider les données avant de les modifier
- ✦ public : accessible partout · protected : classe + sous-classes + package
- ✦ (par défaut, sans mot-clé) : accessible dans le même package uniquement
- ✦ Principe fondamental de la POO : cacher les détails, exposer une interface claire

TP 7 : Gestion des Étudiants

1.Créez une **classe** appelée ***Etudiant*** qui possède les attributs suivants :

- ***nom*** (chaîne de caractères)
- ***age*** (entier)
- ***moyenne*** (double)

2.La classe doit inclure les fonctionnalités suivantes :

- Un ***constructeur*** pour initialiser les attributs.
- Une méthode ***afficherDetails*** qui affiche les informations de l'étudiant.
- Une méthode ***estAdmis*** qui retourne ***true*** si la moyenne est supérieure ou égale à **10**, sinon ***false***.

3.Dans la méthode main, faites les étapes suivantes :

- Créez plusieurs objets de la classe ***Etudiant***.
- Affichez leurs détails.
- Déterminez si chaque étudiant est admis ou non.

Les modificateurs d'accès en Java

Modificateur	Même classe	Même package	Sous-classe	Partout
`private`	Oui	Non	Non	Non
`(par défaut)`	Oui	Oui	Non	Non
`protected`	Oui	Oui	Oui	Non
`public`	Oui	Oui	Oui	Oui

Bonne pratique : attributs toujours private, méthodes public selon le besoin réel (principe du moindre privilège).

this, static et final

1. **this**
   Référence l'objet courant. Utile pour distinguer un attribut d'un paramètre de même nom (this.nom = nom;).

2. **static (attribut)**
   Une seule copie partagée par toutes les instances de la classe (ex : un compteur d'objets créés).

3. **static (méthode)**
   Appelable sans créer d'objet : Classe.methode(). Ne peut pas utiliser this ni les attributs d'instance.

4. **final (variable)**
   Valeur non réassignable après initialisation : constante (souvent combinée avec static : constantes de classe).

5. **final (méthode)**
   Ne peut plus être redéfinie (@Override) par une sous-classe.

6. **final (classe)**
   La classe ne peut pas être héritée (ex : la classe String est final).

Classe de type final

- **Une classe de type final est une classes qui ne peut pas être dérivée.**
- **Exemple de syntaxe:**

La classe CompteSimple peut être déclarée final en écrivant:

```
public final class CompteSimple extends Compte {
  private float decouvert;
  public void afficher() {
    System.out.println("Solde="+solde+"
    Découvert="+decouvert);
}
}
```

Variable & méthode : final

- **Une variable final est une variable dont la valeur ne peut pas changer. Autrement dit, c'est une constante:**
- **Exemple : final double PI=3.14;**
- **Une méthode final est une méthode qui ne peut pas être redéfinie dans les classes dérivées.**
- **Exemple : La méthode verser de la classe suivante ne peut pas être redéfinie dans les classes dérivées car elle est déclarée final**

```
public class Compte {
private int code; protected float solde;
private static int nbComptes;

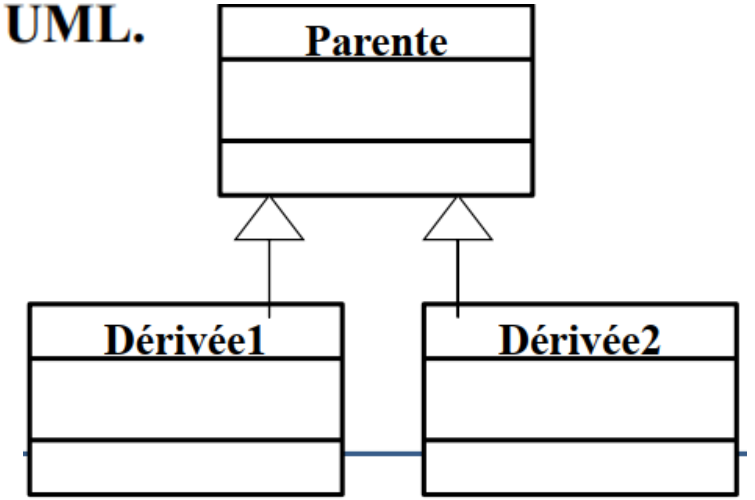
public final void verser(float mt){
   solde+=mt;
}
public void retirer(float mt){
   if(mt<solde) solde-=mt;
}
}
```

Héritage

- **Dans la programmation orientée objet, l'héritage offre un moyen très efficace qui permet la réutilisation du code.**
- **En effet une classe peut hériter d'une autre classe des attributs et des méthodes et non pas les constructeurs.**
- **L'héritage, quand il peut être exploité, fait gagner beaucoup de temps en terme de développement et en terme de maintenance des applications.**
- **La réutilisation du code fut un argument déterminant pour vendre les méthodes orientées objets.**
- **L'héritage en java est traduit par le qualificateur extends.**
- **Toutes les classes héritent implicitement la classe Object.**
- **Une classe ne peut hériter explicitement qu'une seule classe.**

Héritage

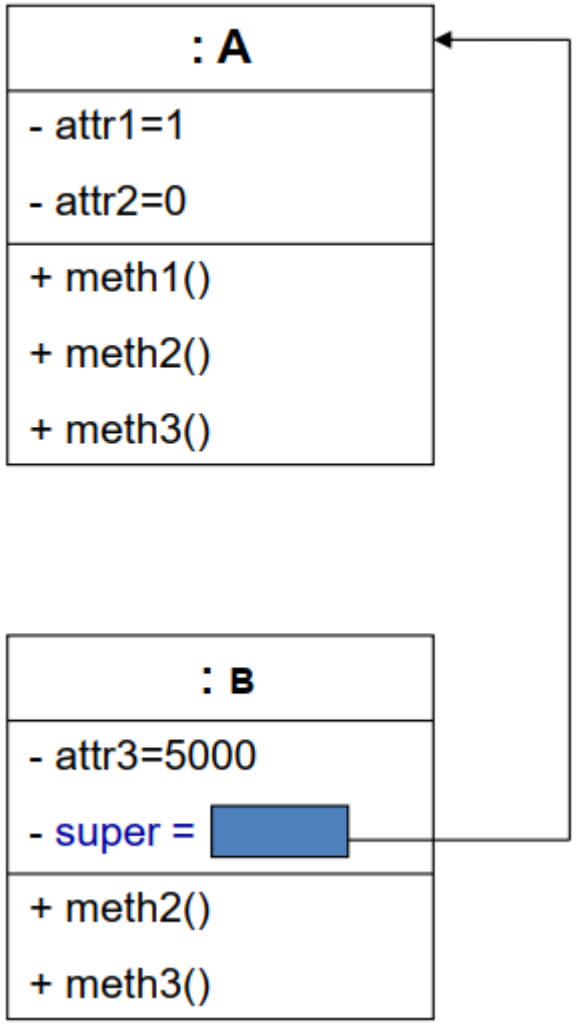
- Si la classe A hérite directement de la classe B alors elle hérite indirectement la classe Object.
- Le constructeur de la classe dérivée peut faire appel au constructeur de la classe parente en utilisant le mot super() suivi de ses paramètres.
- La classe dérivée peut redéfinir les méthodes da la classe parente comme elle peut appeler ces méthodes via le qualificateur super().
- L'héritage et traduit par une association de type généralisation en UML.



Héritage

- **Quand on crée une instance d'une classe, la classe parente est automatiquement instanciée et l'objet de la classe parente est associé à l'objet créé à travers la référence « super » injectée par le compilateur**

- **Lors de l'instanciation, l'héritage entre les classes est traduit par une composition entre un objet de la classe instanciée et d'un objet de la classe parente qui est créé implicitement.**



Héritage avec extends

```
public class Personne {
    protected String nom;
    public Personne(String nom) { this.nom = nom; }
    public void afficher() { System.out.println("Personne:
" + nom); }
}

public class Etudiant extends Personne {
    private String matricule;

    public Etudiant(String nom, String matricule) {
        super(nom);              // appelle le
constructeur de Personne
        this.matricule = matricule;
    }

    @Override
    public void afficher() {
        super.afficher();        // réutilise le
comportement du parent
        System.out.println("Matricule: " + matricule);
    }
}
```

Points clés

- extends : une sous-classe hérite des attributs/méthodes non privés du parent
- Héritage simple en Java (une seule classe mère), mais interfaces multiples possibles
- super(...) appelle le constructeur du parent (obligatoire en première ligne)
- super.methode() permet de réutiliser puis compléter le comportement hérité
- @Override signale explicitement une redéfinition (vérifiée par le compilateur)

Surcharge (overload) vs Redéfinition (override)

Surcharge — overload

- Même nom de méthode, signatures différentes
- Peut se faire dans la même classe
- Choisi à la compilation selon les arguments
- Le type de retour peut différer
- Exemple : moyenne(int[]) et moyenne(int[], double[])

Redéfinition — override

- Même signature exacte que la méthode du parent
- Se fait dans une sous-classe (héritage)
- Choisi à l'exécution selon le type réel de l'objet
- Annotée @Override (bonne pratique, vérifiée par le compilateur)
- Exemple : afficher() redéfinie dans Etudiant

Classes abstraites

- **Une classe abstraite est une classe qui ne peut pas être instanciée.**

- **Une classe abstraite est généralement créée pour en faire dériver de nouvelle classe par héritage.**

- **Exemple:**

```
public abstract class Compte {
        private int code;
        protected float solde;
        private static int nbComptes;
// Constructeurs
// Méthodes
        }
```

Classes abstraites : Méthode abstraite

- **Une méthode abstraite peut être déclarée à l'intérieur d'une classe abstraite.**
- **Une méthode abstraite est une méthode qui n'a pas de définition.**
- **Une méthode abstraite est une méthode qui doit être redéfinie dans les classes dérivées.**

Exemple:

```
public abstract class Compte {
  // Membres
…
// Méthode abstraite
public abstract void afficher();

}
```

```
public class CompteSimple extends Compte  {
  // Membres
…
 public void afficher(){
    System.out.println("Solde="+solde+" Découvert="+decouvert);
 }
}
```

Classes abstraites

```
public abstract class Personne {
    protected String nom;

    public Personne(String nom) { this.nom = nom; }

    public abstract String getRole();   // pas de corps :
à implémenter

    public void afficher() {            // méthode
concrète, héritée telle quelle
        System.out.println(nom + " - " + getRole());
    }
}

public class Etudiant extends Personne {
    public Etudiant(String nom) { super(nom); }

    @Override
    public String getRole() { return "Étudiant"; }
}
```

À retenir

- abstract class : ne peut pas être instanciée directement (pas de new Personne())
- Peut mélanger méthodes concrètes (avec corps) et méthodes abstraites (sans corps)
- Toute sous-classe concrète doit implémenter les méthodes abstraites héritées
- Utile pour factoriser du code commun tout en imposant un contrat

Interfaces

- **Une interface est une sorte de classe abstraite qui ne contient que des méthodes abstraites.**
- **Dans java une classe hérite d'une seule classe et peut implémenter en même temps de plusieurs interface.**
- **Une interface IA peut implémenter une autre interface IB.**
- **Alors toute classe qui implémente l'interface IA implémente aussi l'interface IB.**
- **Une classe qui implémente une interface doit redéfinir toutes les méthodes de cette interface.**

Interfaces

- **Exemple de déclaration d'une interface:**

```
public interface Solvable {
 public void solver();
 public double getSolde();
}
```

- **Exemple d'implementation d'une interface:**

```
public class CompteSimple extends Compte implements Solvable {
private float decouvert;
public void afficher() {
 System.out.println("Solde="+solde+" Découvert="+decouvert);
}
public double getSolde() {return solde;}
public void solver() {this.solde=0;}}
```

Interfaces

```java
public interface Authentifiable {
    boolean seConnecter(String motDePasse);
    default void deconnecter() {
        System.out.println("Déconnexion...");
    }
}

public class Enseignant extends Personne implements
Authentifiable {
    private String motDePasse;

    @Override
    public boolean seConnecter(String motDePasse) {
        return this.motDePasse.equals(motDePasse);
    }
}
```

À retenir

- interface : contrat 100% abstrait par défaut (méthodes sans corps)
- implements : une classe peut implémenter plusieurs interfaces (contrairement à extends)
- Depuis Java 8 : méthodes default (avec corps) et static possibles dans une interface
- Toutes les méthodes d'une interface sont implicitement public
- Utilisée pour définir des comportements transverses (Comparable, Authentifiable…)

Classe abstraite vs Interface : que choisir ?

Classe abstraite

- Héritage simple uniquement (extends)
- Peut avoir des attributs d'instance et un état
- Peut avoir un constructeur
- Mélange méthodes concrètes et abstraites
- "est un" avec une relation forte (Etudiant est une Personne)

Interface

- Implémentation multiple possible (implements)
- Pas d'état (sauf constantes static final)
- Pas de constructeur
- Contrat de comportement (méthodes default possibles)
- "peut faire" — capacité transverse (Authentifiable, Comparable)

Polymorphisme

```
Personne[] personnes = {
    new Etudiant("Amine", "E001"),
    new Enseignant("Rachid", "Java"),
};

for (Personne p : personnes) {
    p.afficher();   // exécute la version redéfinie selon
le type réel
}

// Étudiant.afficher() et Enseignant.afficher()
// sont appelées automatiquement selon l'objet réel
```

Le Polymorphisme permet d’utiliser une même méthode avec des comportement différents selon l’objet concerné.

Ce qu'il faut comprendre

- Une variable de type Personne peut référencer un objet Etudiant ou Enseignant
- La méthode exécutée est celle de la classe réelle de l'objet (liaison dynamique)
- Permet d'écrire du code générique qui fonctionne avec toute sous-classe
- Base de nombreux patrons de conception et de l'architecture DAO/MVC
- instanceof permet, si besoin, de tester le type réel d'un objet

Packages et organisation du code

1

Un package = un espace de noms

package ma.uae.gestion.model; en première ligne du fichier, reflète l'arborescence des dossiers.

2

Éviter les conflits de noms

Deux classes Note peuvent coexister dans des packages différents sans collision.

3

Organisation par responsabilité

model (entités), dao (accès données), service (logique métier), controller, view, util, exception.

4

import pour utiliser une autre classe

import ma.uae.gestion.model.Etudiant; — ou java.util.* pour les classes standard.

5

Convention de nommage

En minuscules, souvent inversé du nom de domaine : ma.uae.gestion (Université Abdelmalek Essaâdi, par exemple).

SYNTHÈSE

Synthèse CH2

- ✓ Une classe décrit attributs + méthodes ; un objet en est une instance
- ✓ Encapsulation : attributs private + accesseurs/mutateurs publics
- ✓ Héritage (extends) : une sous-classe spécialise une classe mère
- ✓ Classe abstraite = base partielle ; interface = contrat de comportement
- ✓ Le constructeur initialise l'objet ; this distingue attribut et paramètre
- ✓ static = partagé par la classe ; final = valeur ou comportement figé
- ✓ Polymorphisme : la méthode exécutée dépend du type réel de l'objet
- ✓ Packages : organiser le code par responsabilité (model, dao, service…)

TP 9 : Gestion des employés d'une entreprise

Créez une classe **Employe** avec les attributs suivants :

- ✓ nom (type String)
- ✓ prenom (type String)
- ✓ salaire (type double)

Cette classe doit comporter :

- ✓ Un constructeur pour initialiser ses attributs.
- ✓ Une méthode afficherDetails() qui affiche les informations de l'employé.

2. Créez une classe **Manager** qui hérite de la classe **Employe** et ajoute un nouvel attribut :

- ✓ prime (type double) représentant une prime de performance.

Cette classe doit comporter :

- ✓ Un constructeur qui initialise tous les attributs (y compris ceux hérités).
- ✓ Une méthode afficherDetails() qui surcharge la méthode de la classe mère pour afficher aussi la prime.

3. Dans une classe principale **Main**, créez un tableau d'employés contenant à la fois des objets de type **Employe** et **Manager**, et affichez leurs détails.